

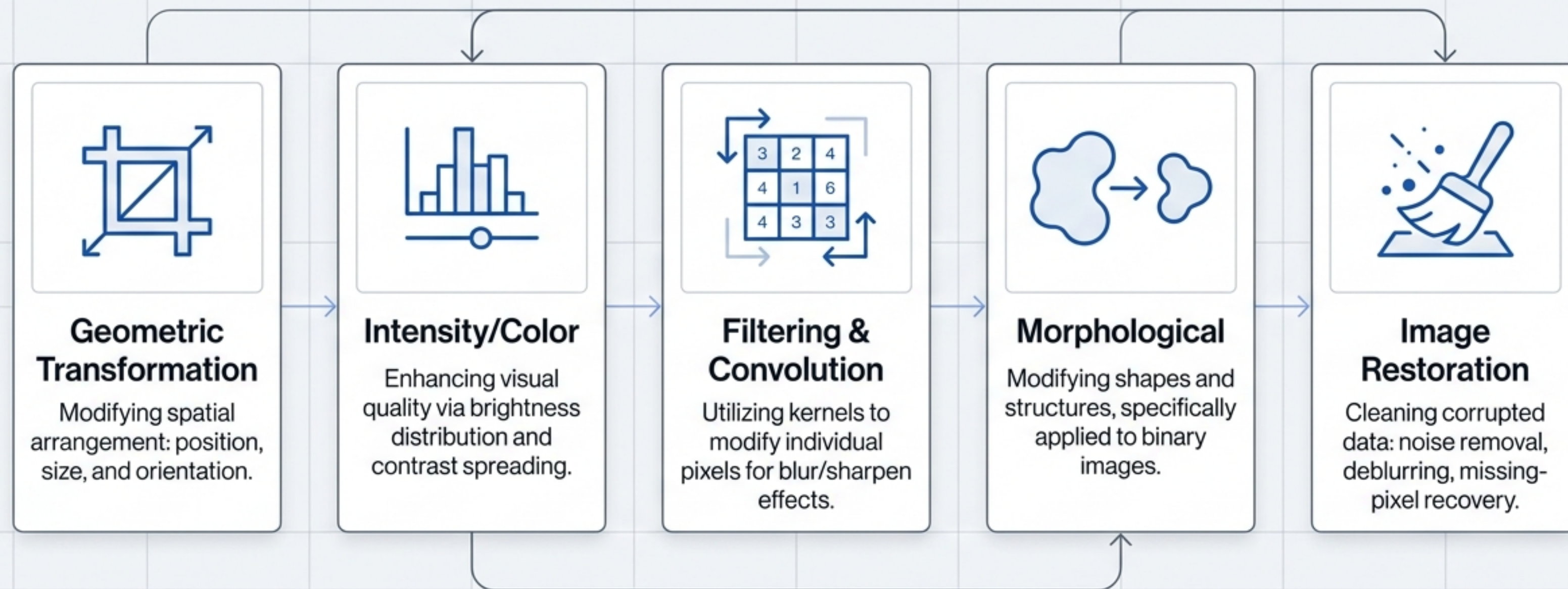


COMPUTER VISION PREPROCESSING

FROM RAW PIXELS TO MACHINE LEARNING PIPELINES

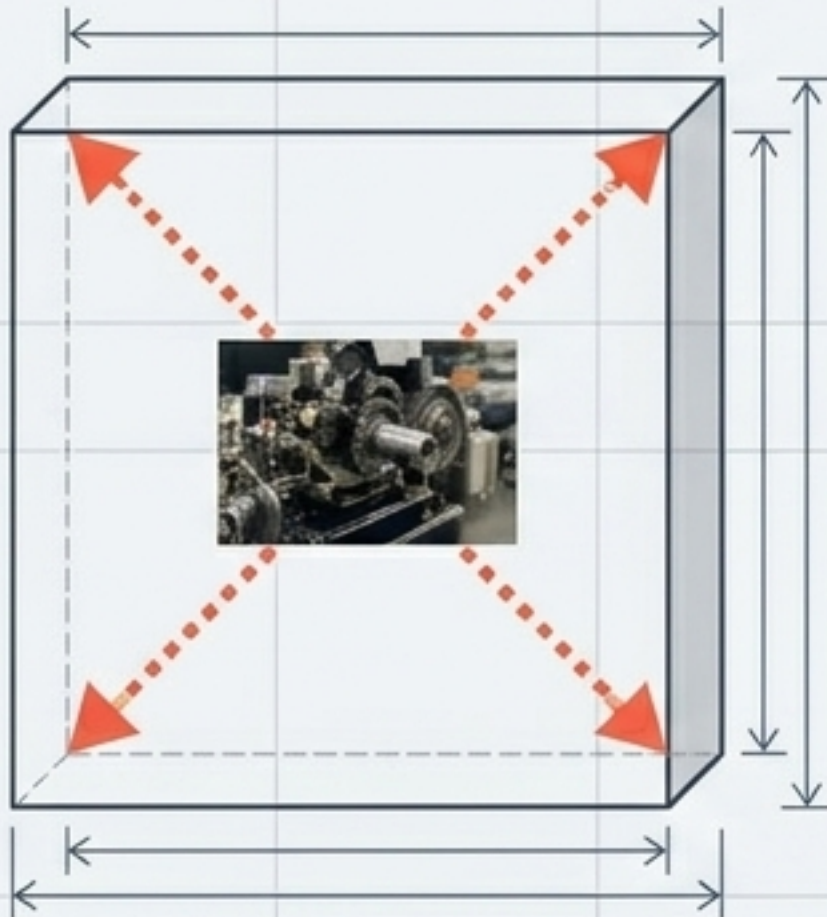
DOC ID: CV-PREP-01.4 // REV: A.2 // DATE: 2024-10-27 // SYSTEM: EDGE-DETECT-ANALYSIS-v4

The Taxonomy of Image Manipulation



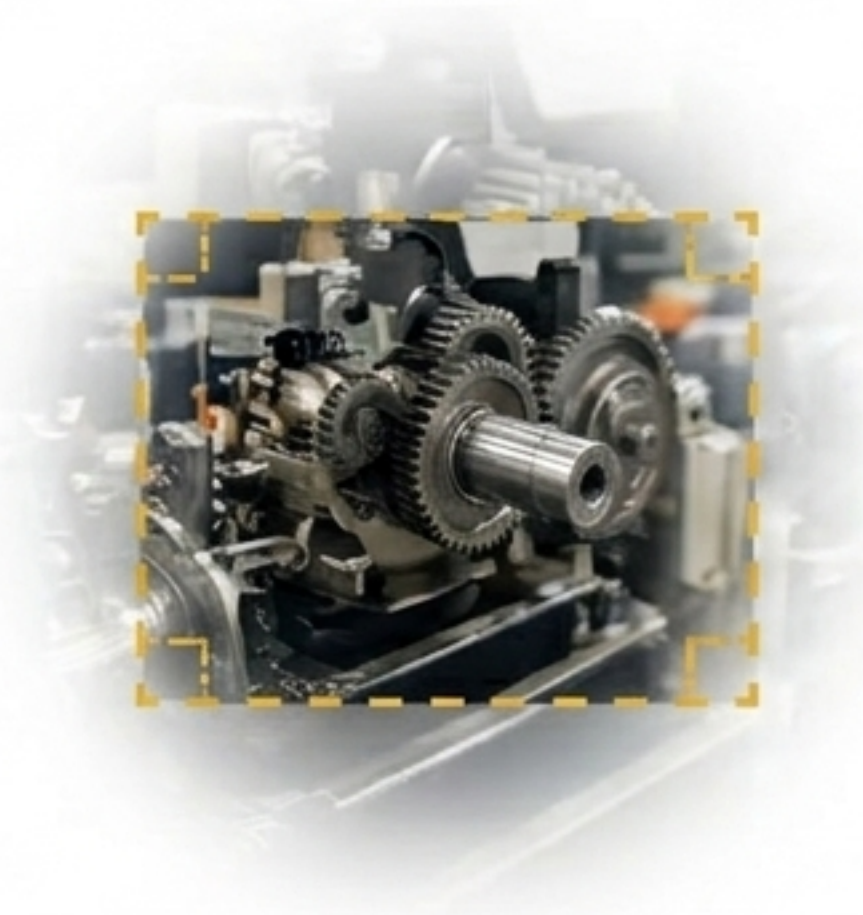
Geometric Structural Changes

Scaling (Resizing)



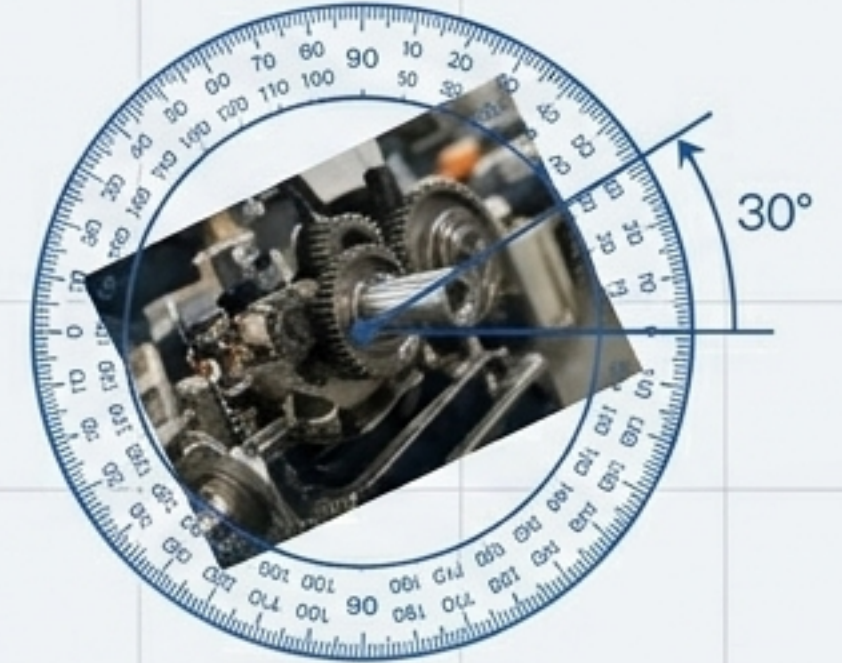
Upscaling or downsampling to standardize image dimensions.

Cropping



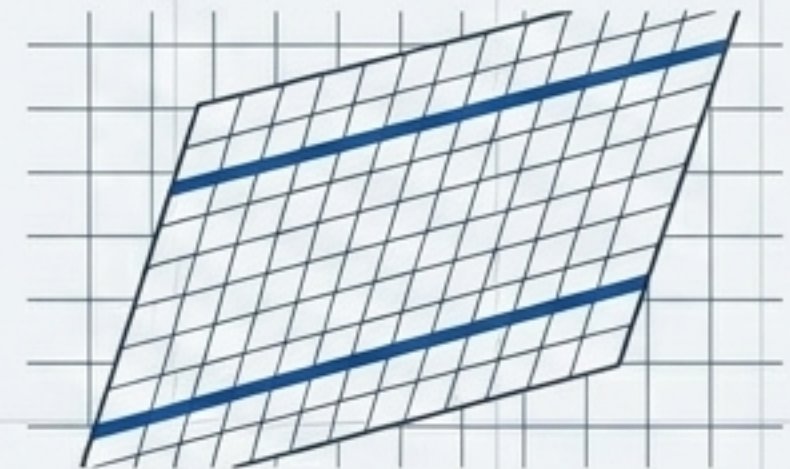
Trimming peripheral edges to focus strictly on the primary subject.

Rotation



Reorienting the image by a specific degree angle around a central pivot point.

Affine Transformation



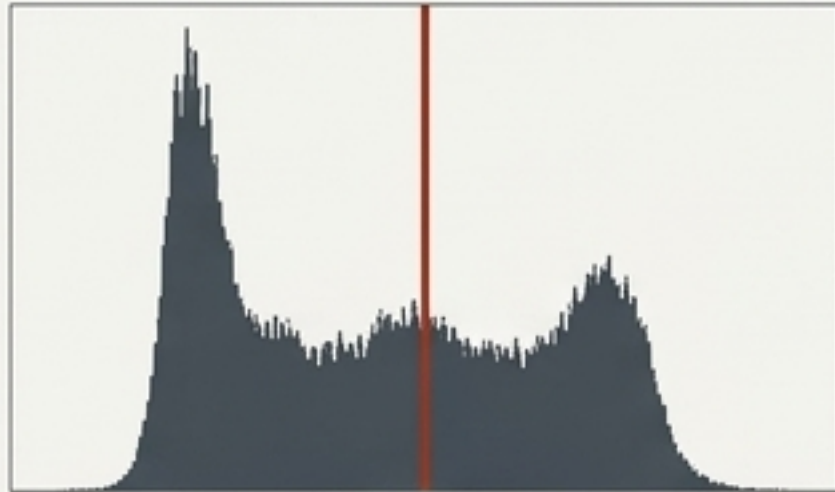
Linear mapping preserving lines and parallelism (scaling, rotation, translation, shearing).

The Mathematics of Interpolation

Method	Sampling Logic	Speed vs. Quality	Ideal Use Case
Nearest Neighbor	 Assigns value of closest pixel.	Fastest computation.	Best for pixel art / fast standardizing.
Bilinear Interpolation	 Calculates values based on 4 nearest pixels.	Balanced.	The standard for photography and ML preprocessing.
Bicubic Interpolation	 Uses a 16-pixel neighborhood.	Slower, highest quality.	Superior results for high-quality upscaling.

Feature Extraction: Thresholding Strategies

Simple Thresholding



Constant Global Threshold



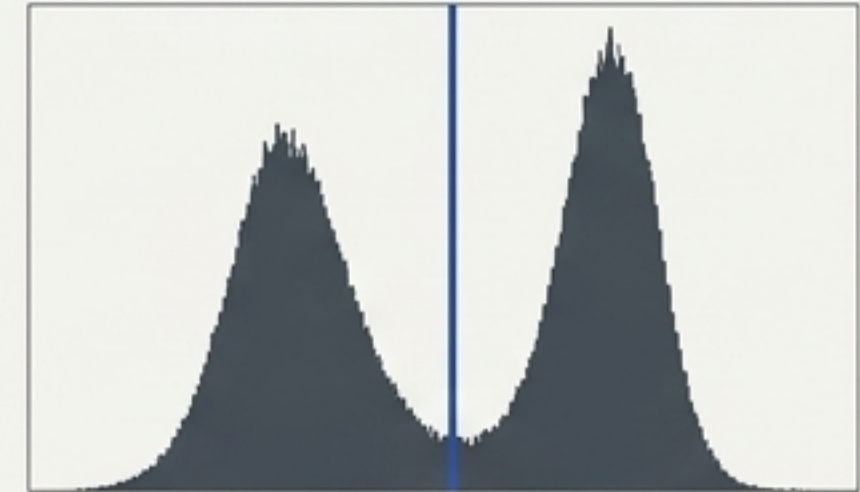
Compared to a global constant value
(above = white, below = black).

Adaptive Thresholding



Calculates different localized thresholds
to handle varying lighting conditions.

Otsu's Thresholding

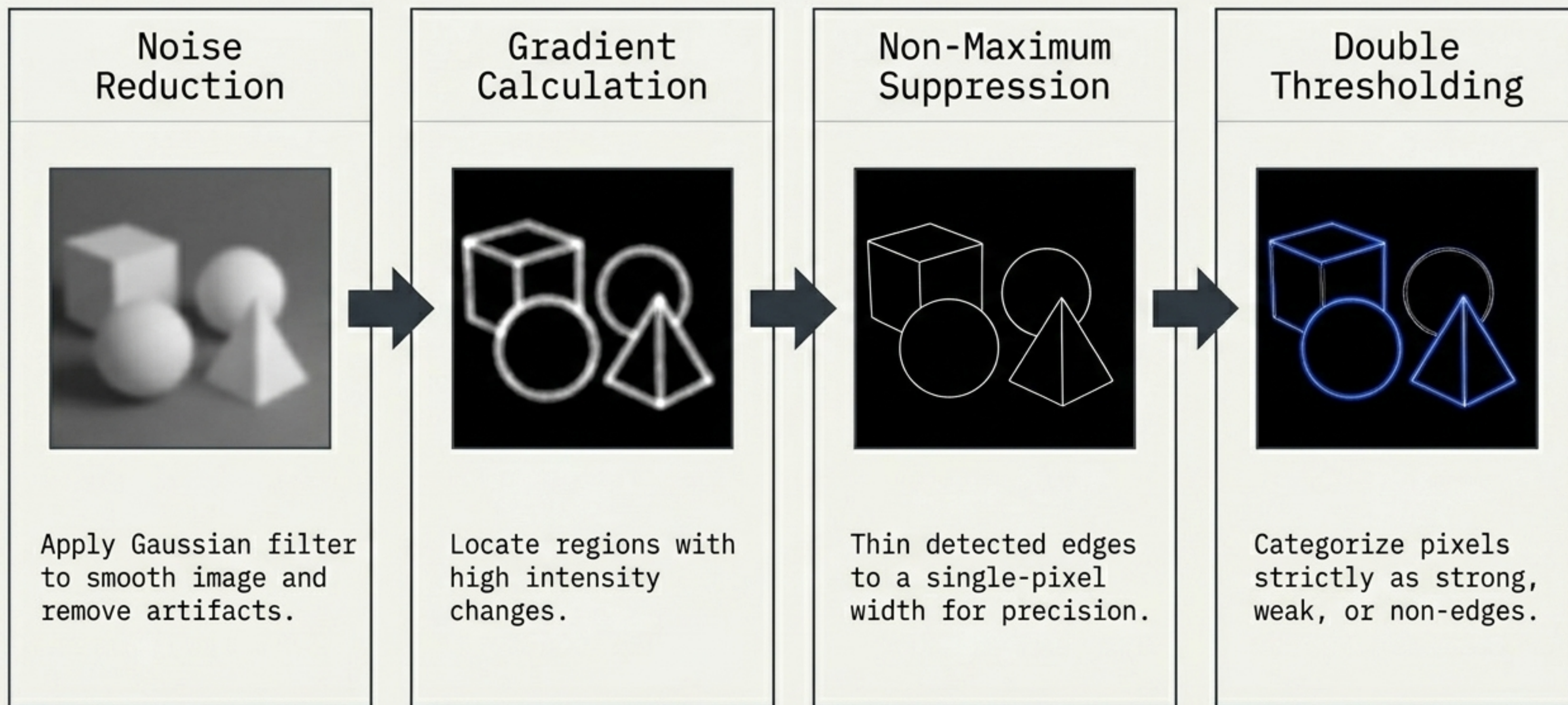


Optimal Global Threshold



Automatically determines optimal global
threshold by maximizing inter-class variance.

The Canny Edge Detection Pipeline



Practical Implementation: Spatial Formatting

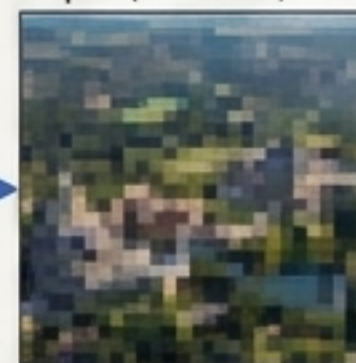
```
high_res = cv2.resize(img, (1024, 1024), interpolation=cv2.INTER_CUBIC)
std_data = cv2.resize(img, (224, 224), interpolation=cv2.INTER_NEAREST)

start_x, start_y = w//4, h//4
end_x, end_y = 3*w//4, 3*h//4
roi = img[start_y:end_y, start_x:end_x]
```

High-Resolution Output (1024x1024, INTER_CUBIC)



Standard ML Input (224x224, INTER_NEAREST)



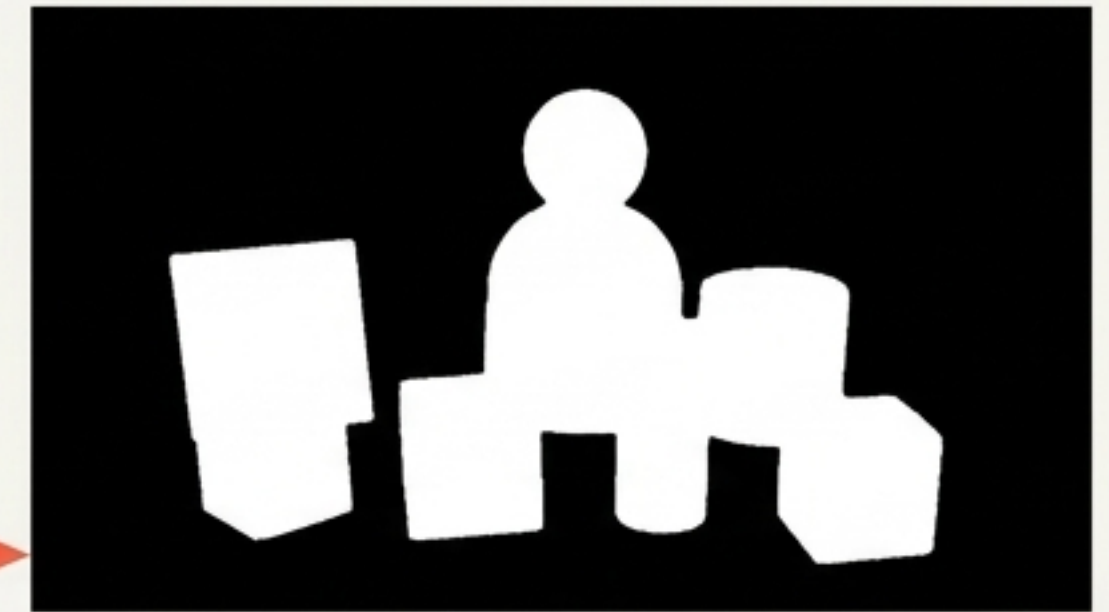
Region of Interest (ROI) Crop



Practical Implementation: Masking & Edges

Otsu Auto-Thresholding

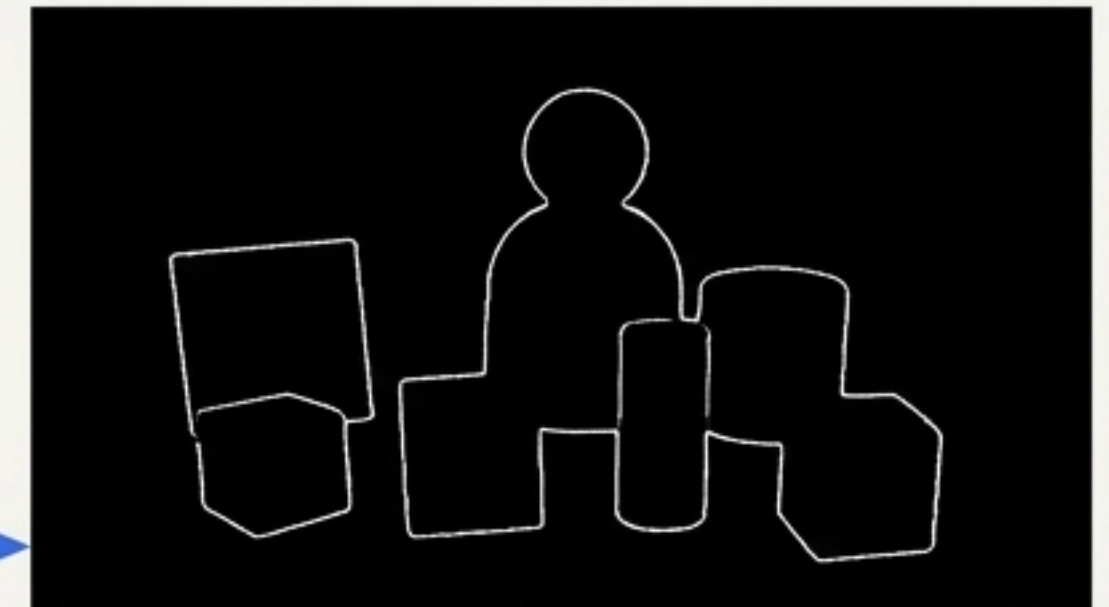
```
_, otsu = cv2.threshold(gray, 0, 255,  
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```



Otsu Binary Mask Output

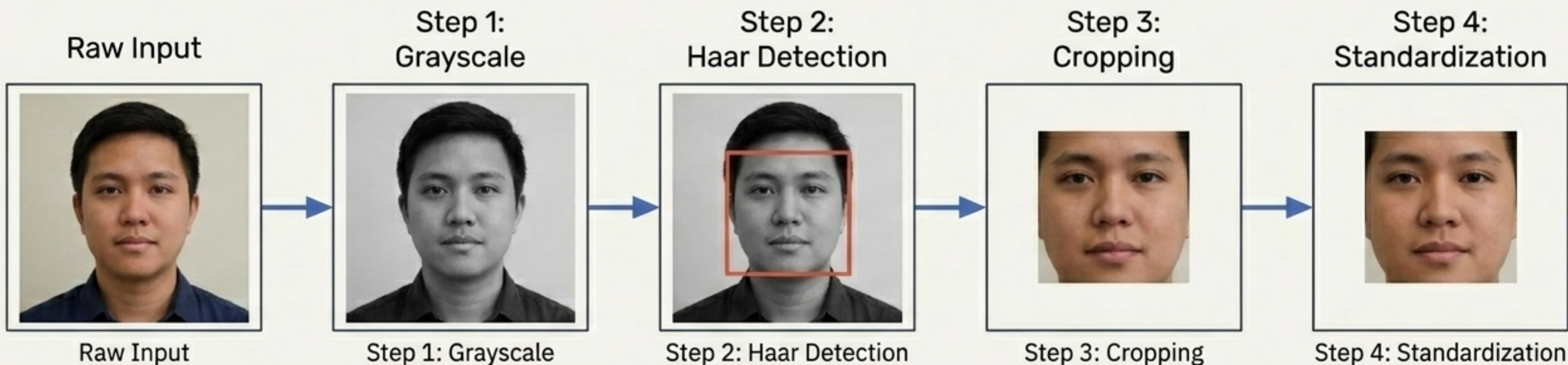
Canny Pipeline Execution

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)  
edges = cv2.Canny(blur, 100, 200)
```



Canny Edges Output

The Automated ML Preprocessing Pipeline



```
faces = face_cascade.detectMultiScale(gray, 1.1, 4)
for (x, y, w, h) in faces:
    face = img[y:y+h, x:x+w]
    std_face = cv2.resize(face, (128, 128))
    cv2.imwrite('final_face_input.jpg', std_face)
```

This short block autonomously orchestrates identity verification system inputs.

CV to OpenCV Rosetta Stone

Concept	Goal	OpenCV Function
Scaling (Bicubic)	High-quality upsampling	<code>cv2.resize(img, dim, interpolation=cv2.INTER_CUBIC)</code>
Cropping / ROI	Isolate subject focus	<code>img[start_y:end_y, start_x:end_x]</code>
Otsu's Threshold	Automated optimal segmentation	<code>cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)</code>
Canny Edge	Structural boundary map	<code>cv2.Canny(image, low_thresh, high_thresh)</code>
Object Detection	Locate structural patterns	<code>cascade.detectMultiScale(image, scaleFactor, minNeighbors)</code>